ASTRAFORGE AEROSPACE · YNOV BACHELOR 2

ORBITAL COMMAND

Jour 4 bis — Router & Ground Link

DYLAN · MARDI 8 SEPTEMBRE 2026 · 4 HEURES · 20 % THÉORIE · 80 % PRATIQUE

BUTS DU COURS

- Ouvrir les modules Flotte, Missions, Contrôle et Alertes
- Ouvrir une ressource par son identifiant
- Distinguer calcul, événement et effet
- Appeler une API REST et représenter les états réseau
- Extraire un hook réutilisable

SÉQUENCE 1 · BRIEFING

ROUTER & GROUND LINK

Jour 4 bis — Une URL pour chaque destination, un lien sol qui ne ment pas

Une URL pour chaque destination. Un réseau qui n'est jamais « juste une requête ».

SÉQUENCE 1 · RAPPEL

Où on en est

Jours 1 et 2 — modèles typés, registre de flotte interactif

- **Jour 1** — TypeScript : `type`, `interface`, union, narrowing
- **Jour 2** — React : composants, props, `useState`, filtre, Tailwind

La flotte marche. Elle vit encore sur **un seul écran**, avec des mocks locaux.

type et interface

Un contrat pour le domaine, pas un slogan

type-interface.ts

```ts
interface VehicleBase {
  readonly id: string
  name: string
  status: VehicleStatus
}

type VehicleStatus = "grounded" | "ready" | "in-flight"
```

`id` est `readonly`. Le statut est une union de libellés, pas une `string` libre.

Union discriminante

Un champ kind pour savoir quelle forme on a

discriminated-union.ts

```ts
type Vehicle =
  | { kind: "crewed"; crewCapacity: number }
  | { kind: "cargo"; payloadKg: number }
  | { kind: "autonomous"; autonomyLevel: number }
```

Crewed, cargo et autonome ne partagent pas les mêmes métriques. `kind` les sépare.

Narrowing

Après le if, TypeScript connaît le variant

narrowing.ts

```ts
function describeVehicle(vehicle: Vehicle) {
  if (vehicle.kind === "cargo") {
    return `${vehicle.payloadKg} kg de charge utile`
  }
}
```

Pas de pile de `?.` : on resserre le type, puis on lit le champ propre à la catégorie.

TSX typé

Les données du domaine deviennent une carte

vehicle-card.tsx

```tsx
type VehicleCardProps = {
  vehicle: Vehicle
}

export function VehicleCard({ vehicle }: VehicleCardProps) {
  return <article>{vehicle.name}</article>
}
```

Props = contrat du composant

StatusBadge ne devine pas le statut, il le reçoit

status-badge-props.tsx

```tsx
interface StatusBadgeProps {
  status: VehicleStatus
}

function StatusBadge({ status }: StatusBadgeProps) {
  return <span>{status}</span>
}

<StatusBadge status={vehicle.status} />
```

children = un emplacement

Le panel porte la structure, pas le contenu métier

panel-children.tsx

```tsx
interface PanelProps {
  title: string
  children: React.ReactNode
}

function Panel({ title, children }: PanelProps) {
  return (
    <section>
      <h2>{title}</h2>
      {children}
    </section>
  )
}

<Panel title="Fleet Status">
  <FleetGrid vehicles={vehicles} />
</Panel>
```

useState mémorise entre deux rendus

Modifier une variable locale ne redessine rien

usestate.tsx

```tsx
const [selectedStatus, setSelectedStatus] =
  useState<VehicleStatus | "all">("all")

setSelectedStatus("maintenance")
```

Chaque rendu voit un instantané. Pour changer l'écran, on appelle le setter.

Mise à jour immuable

React a besoin d'une nouvelle référence

immutable-update.tsx

```tsx
// à éviter
vehicles[0].status = "maintenance"
setVehicles(vehicles)

// à privilégier
setVehicles((current) =>
  current.map((vehicle) =>
    vehicle.id === vehicleId
      ? { ...vehicle, status: "maintenance" }
      : vehicle
  )
)
```

key stable = vehicle.id

L'index de tableau n'identifie pas un véhicule

list-keys.tsx

```tsx
// à privilégier
vehicles.map((vehicle) => (
  <VehicleCard
    key={vehicle.id}
    vehicle={vehicle}
  />
))

// à éviter
key={index}
```

Le filtre se calcule au rendu

Pas un second état, pas un useEffect

derived-filter.tsx

```tsx
// à éviter
const [vehicles, setVehicles] = useState(...)
const [filteredVehicles, setFilteredVehicles] =
  useState(...)

// à privilégier
const filteredVehicles = vehicles.filter(...)
```

On y revient aujourd'hui : un effet n'est pas l'endroit pour filtrer ou coller deux chaînes.

Champ contrôlé

React est la source de vérité de la recherche

controlled-input.tsx

```tsx
const [search, setSearch] = useState("")

<input
  value={search}
  onChange={(event) =>
    setSearch(event.target.value)
  }
/>
```

SÉQUENCE 1 · RAPPEL

Ce qui manque encore

On a une flotte typée, découpée, filtrable, habillée.

Il manque :

- **plusieurs destinations** — une URL par module, une fiche par ressource
- **le lien sol** — `fetch`, états réseau, un hook réutilisable

Tout le reste du jour part de ce dépôt. On n'efface pas le jour 2, on l'ouvre.

SÉQUENCE 1 · BRIEFING

Mission du jour

Ouvrir le centre de contrôle, puis le brancher au sol

- créer les modules Flotte, Missions, Contrôle et Alertes
- ouvrir une ressource par son identifiant
- distinguer calcul, événement et effet
- appeler une API REST
- extraire un hook réutilisable

SÉQUENCE 2 · PENSER EN ROUTES

Une SPA n'est pas une page unique

Le navigateur charge une application, puis le routeur associe l'URL à une vue

- `/fleet` – registre de flotte
- `/missions` – liste des missions
- `/control` – centre de contrôle
- `/alerts` – journal des alertes

L'URL est un état partageable

Une bonne URL survit au rafraîchissement et au partage

- revenir avec le bouton précédent
- actualiser sans perdre le contexte
- partager une ressource
- créer un favori
- reproduire un bug

Le routeur à la racine

Le routeur fournit le contexte de navigation à toute l'application

browser-router.tsx

```
<BrowserRouter>
  <App />
</BrowserRouter>
```

Déclarer les destinations

Une route relie un chemin à un élément React

routes.tsx

```tsx
<Routes>
  <Route path="/fleet" element={<FleetPage />} />
  <Route path="/missions" element={<MissionsPage />} />
</Routes>
```

Link, pas un bouton maquillé

Choisir le bon élément améliore clavier, accessibilité et compréhension

link.tsx

```tsx
<Link to="/missions">Voir les missions</Link>
```

- `Link` change de destination
- un `button` déclenche une action

NavLink connaît la destination active

Le module actif doit être identifiable autrement que par la couleur seule

navlink.tsx

```tsx
<NavLink to="/fleet">
  {({ isActive }) => isActive ? "Flotte •" : "Flotte"}
</NavLink>
```

SÉQUENCE 2 · INCIDENT

Incident — la fausse navigation

L'équipe a utilisé quatre boutons et un useState("fleet")

Remplacer ce faux routage sans perdre les composants existants.

TP 1 — Open the Modules

Plan de routes + quatre destinations, une navigation persistante

Créer :

- les constantes de chemins
- `FleetPage`, `MissionsPage`, `ControlPage`, `AlertsPage`
- une navigation persistante
- les coquilles `/fleet/:vehicleId`, `/missions/:missionId` et `*`

Chaque page possède un titre unique et une URL directe fonctionnelle. F5 conserve la destination.

Un chemin, plusieurs véhicules

:vehicleId représente un segment variable de l'URL

vehicle-route.tsx

```tsx
<Route
  path="/fleet/:vehicleId"
  element={<VehicleDetailPage />}
/>
```

Lire et vérifier le paramètre

{ "Le paramètre est une entrée externe": "il peut être absent ou invalide" }

use-params.tsx

```tsx
const { vehicleId } = useParams<{ vehicleId: string }>()
const vehicle = vehicles.find((item) => item.id === vehicleId)
```

Une ressource absente est un cas normal

Ni écran vide, ni erreur JavaScript, ni redirection silencieuse

resource-not-found.tsx

```tsx
if (!vehicle) {
  return <ResourceNotFound resource="véhicule" />
}
```

Navigation programmatique

Après une action, ouvrir le contrôle de la mission choisie

use-navigate.tsx

```
const navigate = useNavigate()
navigate(`/fleet/${vehicleId}`)
```

Utilisez cette navigation seulement après une action, pas pour remplacer tous les liens.

SÉQUENCE 3 · LOST IN SPACE

Deux absences différentes

Ces deux erreurs peuvent partager un ton, mais pas le même message

- `/fleet/unknown` — la route existe, la ressource non
- `/pizza-on-mars` — aucune route ne correspond

Route attrape-tout

La page doit expliquer ce qui s'est passé et proposer une destination sûre

not-found-route.tsx

```
<Route path="*" element={<NotFoundPage />} />
```

SÉQUENCE 3 · TP 2

TP 2 — Resource Detail

Fiches véhicule et mission + 404 utile

- identité, statut, métriques de catégorie
- objectif, fenêtre, véhicule assigné, phase
- `ResourceNotFound` réutilisable
- 404 « Signal introuvable » distincte
- action « Ouvrir le contrôle »

Trois endroits pour la logique

- **rendu** : coller deux prénoms, filtrer une liste
- **clic** : envoyer un message, incrémenter un compteur
- **effet** : parler au navigateur ou au réseau (titre, minuteur, `fetch`)

Si ça ne sort pas de React, vous n'avez probablement pas besoin de `useEffect`.

Un nom complet se calcule

Pas besoin d'un effet pour coller deux chaînes

derived-name.tsx

```tsx
// à éviter
useEffect(() => {
  setFullName(firstName + " " + lastName)
}, [firstName, lastName])

// à privilégier
const fullName = firstName + " " + lastName
```

Ada + Lovelace = Ada Lovelace, tout de suite, au même rendu.

Un clic reste un clic

Pas d'état shouldSend surveillé par un effet

click-not-effect.tsx

```tsx
// à éviter : le clic pose un drapeau, l'effet envoie
useEffect(() => {
  if (shouldSend) sendMessage()
}, [shouldSend])

// à privilégier : le clic envoie
function handleClick() {
  sendMessage()
}
```

SÉQUENCE 4 · EFFETS

L'effet, c'est pour l'extérieur

Titre d'onglet, horloge, requête HTTP

React a fini de dessiner. Ensuite seulement, l'effet s'exécute.

Exemples simples :

- écrire `document.title`
- démarrer un `setInterval`
- appeler `fetch`

Et toujours : arrêter ce travail quand le composant part ou quand la valeur change.

Exemple 1 — le titre de l'onglet

use-effect-title.ts

```ts
useEffect(() => {
  document.title = "Bonjour " + name
}, [name])
```

`name` change → React refait le titre. `name` inchangé → rien.

Exemple 2 — un minuteur

Démarrer, puis couper à la sortie

use-effect-timer.ts

```ts
useEffect(() => {
  const id = setInterval(() => {
    setSeconds((s) => s + 1)
  }, 1000)

  return () => clearInterval(id)
}, [])
```

Sans `return () => clearInterval(id)`, le compteur continue après avoir quitté la page.

Le tableau à la fin n'est pas décoratif

- `[name]` – relancer si le nom change
- `[]` – une seule fois (le minuteur)
- **oublier** `name` – l'onglet reste coincé sur « Ada »
- **mettre** `count` **alors que l'effet fait** `setCount` – le compteur s'emballe

Trop étroit / trop large

missing-dep.ts

```tsx
// trop étroit : name change, le titre reste "Ada"
useEffect(() => {
  document.title = name
}, [])

// trop large : boucle, count grimpe tout seul
useEffect(() => {
  setCount(count + 1)
}, [count])
```

En développement, React monte deux fois

Strict Mode (dev seulement) : monte → nettoie → remonte.

- **minuteur bien écrit** : un seul intervalle à la fin (le premier a été coupé)
- **minuteur sans cleanup** : deux intervales, le compteur saute de 2
- **boucle** : ça n'arrête jamais, même en production

Pour un `fetch`, c'est la même idée : au cleanup on appelle `abort()`.

Une réponse HTTP n'est pas encore une donnée fiable

fetch-unknown.ts

```ts
const response = await fetch("/api/vehicles")
if (!response.ok) throw new Error(`HTTP ${response.status}`)
const data: unknown = await response.json()
```

Les interfaces TypeScript disparaissent à l'exécution.

Typer la frontière

Séparer :

- le type reçu de l'API
- la validation ou vérification minimale
- la transformation en modèle d'interface
- l'affichage

Cette frontière absorbe les différences de nommage, format et unité.

Service minimal

get-vehicles.ts

```ts
export async function getVehicles(signal?: AbortSignal) {
  const response = await fetch("/api/vehicles", { signal })
  if (!response.ok) throw new Error("Ground link unavailable")
  return response.json() as Promise<VehicleDto[]>
}
```

L'assertion ne valide pas la donnée : elle documente seulement une attente.

SÉQUENCE 5 · TP 3

TP 3 — Establish Ground Link

Remplacer les mocks, avec un useEffect correctement cadré

- `GET /api/vehicles.json`
- `GET /api/missions.json`

Contraintes — services dédiés, erreurs HTTP, requêtes annulables, dépendances stables, pas de boucle, pas de filtre dans un effet

API bizarre

Écrire l'adaptateur sans contaminer les composants

vehicle-dto.json

```
{
  "vehicle_id": 7,
  "fuel_ratio": 0.82,
  "state": "GO"
}
```

Résultat attendu dans l'UI : `{ id: "7", fuelPercent: 82, status: "ready" }`

SÉQUENCE 6 · ÉTATS RÉSEAU

Le succès n'est qu'un état parmi cinq

IDLE LOADING SUCCESS EMPTY ERROR

idle → loading → success, empty ou error. L'interface doit rendre chaque état volontairement.

Loading — montrer que le système travaille

Un bon chargement :

- préserve la structure attendue
- ne fait pas clignoter inutilement
- est annoncé si l'attente est significative
- évite de simuler une donnée réelle

Empty n'est pas Error

- **empty** : la requête a réussi, aucune mission ne correspond
- **error** : le résultat n'a pas pu être obtenu

Les actions proposées diffèrent : modifier le filtre ou réessayer la connexion.

SÉQUENCE 6 · TP 4

TP 4 — Signal Lost

Cinq scénarios, plus une donnée partielle

1. 2 secondes de latence
2. HTTP 500
3. ressource 404
4. tableau vide
5. retour au succès

Ajouter skeleton, message utile et bouton de retry.

SÉQUENCE 6 · INCIDENT

Données partielles

Hermes arrive sans valeur de carburant

L'écran doit continuer à fonctionner.
Jamais inventer 0 %.

Afficher « donnée indisponible » et conserver une erreur visible dans le journal technique.

Un hook, c'est une fonction qui commence par use

Comme `useState`, mais à vous.

- on l'appelle en haut du composant (pas dans un `if`)
- à l'intérieur : `useState`, `useEffect`, etc.
- ça **n'affiche rien** : ça rend une valeur (ou une action)

But : ne pas recopier le même effet dans trois fichiers.

Exemple 1 — useDocumentTitle

L'effet du titre, sorti du composant

use-document-title.ts

```ts
function useDocumentTitle(title: string) {
  useEffect(() => {
    document.title = title
  }, [title])
}

useDocumentTitle("Flotte")
```

Dans la page : `useDocumentTitle("Flotte")`. L'effet vit dans le hook.

Exemple 2 — useSeconds

L'état + le minuteur + le cleanup, ensemble

use-seconds.ts

```ts
function useSeconds() {
  const [seconds, setSeconds] = useState(0)
  useEffect(() => {
    const id = setInterval(
      () => setSeconds((s) => s + 1),
      1000,
    )
    return () => clearInterval(id)
  }, [])
  return seconds
}
```

Dans la page : `const seconds = useSeconds()` — plus de `setInterval` visible.

Ensuite, même geste pour la flotte

`useDocumentTitle` parle à l'onglet.
`useSeconds` parle à l'horloge.
`useVehicles` parle au réseau.

- le hook lance le `fetch`, l'annule, stocke le résultat
- la page affiche : chargement, liste, vide, ou erreur
- `useMissions` : même forme, autre URL

Côté page, ça ressemble à ça

use-vehicles-same.ts

```
// même idée, autre extérieur : le réseau
const { status, data, reload } = useVehicles()
```

Le TP 5 consiste à écrire ce hook, pas à le recopier depuis la slide.

TP 5 — Hook Extraction

Créer useVehicles et useMissions — checkpoint session-04

- état distant typé
- requête initiale
- annulation
- `reload()`
- aucune dépendance manquante
- interface simple à consommer

Definition of Done

- quatre modules accessibles, navigation sémantique
- deux routes dynamiques, ressource absente et 404 utiles
- flotte et missions viennent de l'API
- effets réservés aux synchronisations externes
- dépendances complètes, cleanup présent
- loading, empty, error et success visibles
- retry fonctionnel, hook personnalisé extrait

SÉQUENCE 8 · CLÔTURE

Démo réseau dégradé

Chaque groupe montre en 60 secondes

1. un chargement lent
2. une panne
3. une récupération
4. une donnée partielle correctement traitée

Commit de mission

Tag attendu — session-04

commit.txt

```
feat(session-04): connect ground API and handle remote states
```

SÉQUENCE 8 · TICKET DE SORTIE

Répondez en trois lignes

1. quelle différence entre `Link` et `useNavigate` ?
2. quel code ne méritait finalement pas d'effet ?
3. quelle différence entre `empty` et `error` ?

SÉQUENCE 8 · SUITE

Mission Planner

Demain, saisir des ordres et empêcher les lancements impossibles

- formulaires contrôlés
- validation accessible
- `useReducer`
- Context opérationnel
- interruption confirmée

Les routes sont ouvertes. Le lien sol fonctionne. Il faut maintenant partager un état opérationnel complexe.